Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251188
Nama Lengkap	ANGELA REVALINE SETIAWAN
Minggu ke / Materi	11 / Tipe Data List

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Link Github : https://github.com/angelarevalines/71251188_valine.git

1. Sifat-sifat List

- List dalam Python merupakan rangkaian nilai-nilai yang dapat diakses menggunakan satu nama tunggal.
- Jika string adalah rangkaian dari karakter-karakter, sedangkan list bisa berisi karakter, integer, float, maupun tipe data lainnya.
- Rangkaian nilai-nilainya dapat dituliskan di dalam []

```
nilai_ujian = [80,95,99,86,88,97,95]
nama_mahasiswa = ['linn', 'ceii', 'ciaciol', 'vii']
nilai_campuran = ['Python', 88, 23.7, True]
list_dalam_list = [88, [23, 7], 29]
```

Gambar 11.1 : contoh tipe data list.

- List bersifat **mutable**, sedangkan String bersifat **immutable**.
- **Mutable** berarti nilainya dapat diubah secara langsung

```
laprak.py > ...
nilai = [5,25,50,75]
print (f"sebelum diubah = {nilai}")
nilai[2] = 65
print (f"setelah diubah = {nilai}")

TERMINAL ...  Python
PS C:\Users\katar\Downloads\minggu11> & C:/Users/kat
ar/AppData/Local/Python/pythoncore-3.14-64/python.ex
e c:/Users/katar/Downloads/minggu11/laprak.py
sebelum diubah = [5, 25, 50, 75]
setelah diubah = [5, 25, 65, 75]
PS C:\Users\katar\Downloads\minggu11>
```

Gambar 11.2 : list bersifat mutable.

- Jika ada 2 string yang isinya sama maka akan mengembalikan nilai “True” sedangkan jika dalam list isinya sama maka akan mengembalikan nilai “False”

```
laprak.py > ...
a = "valine"
b = "valine"
print (a is b)

PROBLEMS  TERMINAL ...
PS C:\Users\katar\Down
ar/AppData/Local/Pytho
e c:/Users/katar/Downl
True
PS C:\Users\katar\Down
```

```
laprak.py > ...
a = [2,3,7]
b = [2,3,7]
print (a is b)

PROBLEMS  TERMINAL ...
PS C:\Users\katar\Downloa
ar/AppData/Local/Python/p
e c:/Users/katar/Download
False
PS C:\Users\katar\Downloa
```

Gambar 11.3 dan 11.4 : jika string sama = True sedangkan jika list sama = False.

2. Mengakses dan Mengubah isi List

Mengakses elemen list dapat dengan cara menggunakan indeksnya. Kita juga dapat mengubah isi dari list tersebut.

```
laprak.py > ...
1   list = ["informatika", "ukdw", "yogyakarta"]
2   print(list[1])
3   print(list[-1])
4   print(list[1:5])
5   print(list[:2])

PROBLEMS   OUTPUT   TERMINAL   ...   Python

PS C:\Users\katar\Downloads\minggu11> & C:/Users/katar
pythoncore-3.14-64/python.exe c:/Users/katar/Downloads
ukdw
yogyakarta
['ukdw', 'yogyakarta']
['informatika', 'ukdw', 'yogyakarta']
```

```
laprak.py > ...
1   list = ["informatika", "ukdw", "yogyakarta"]
2   print(list[1])
3   print(list[-1])
4   print(list[1:5])
5   print(list[:2])
6
7   list[2] = "fti"
8   print(list)

PROBLEMS   OUTPUT   TERMINAL   ...   Python

PS C:\Users\katar\Downloads\minggu11> & C:/Users/kata
pythoncore-3.14-64/python.exe c:/Users/katar/Download
ukdw
yogyakarta
['ukdw', 'yogyakarta']
['informatika', 'ukdw']
['informatika', 'ukdw', 'fti']
```

Gambar 11.5 dan Gamber 11.6 : contoh mengakses dan mengubah elemen list menggunakan indeks.

```
laprak.py > ...
1   list = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
2   list[1:3] = ["blackcurrant", "watermelon"]
3   print(list)

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\katar\Downloads\minggu11> & C:/Users/katar/AppData/Local/Pyth
['apple', 'blackcurrant', 'watermelon', 'orange', 'kiwi', 'mango']
```

Gambar 11.7 : contoh mengubah elemen list lebih dari satu elemen.

3. Fungsi-fungsi Untuk List

Penambahan list juga bisa dilakukan dengan menggunakan operator **+** dan pengulangan menggunakan operator *.

```
laprak.py > ...
1   list1 = [1,2,3,4,5]
2   list2 = [6,7,8]
3
4   list3 = list1 + list2
5   list4 = list1 * 2
6
7   print(list3)
8   print(list4)

PROBLEMS   OUTPUT   TERMINAL   ...

PS C:\Users\katar\Downloads\mingg
pythoncore-3.14-64/python.exe c:/
[1, 2, 3, 4, 5, 6, 7, 8]
[1, 2, 3, 4, 5, 1, 2, 3, 4, 5]
```

Gambar 11.8 : contoh penggunaan operator + dan * dalam list.

Beberapa method yang Python berikan tentang list :

a) append = untuk menambahkan elemen baru, lalu dianggap sebagai kesatuan objek pada bagian akhir list.

```
laprak.py > ...
1   nama = ["valine", "ceii", "ciaciol", "vii"]
2   nama.append(["kimi", "karin"])
3   print(nama)

PROBLEMS   OUTPUT   TERMINAL   ...   Python

PS C:\Users\katar\Downloads\minggu11> & C:/Users/katar/A
pythoncore-3.14-64/python.exe c:/Users/katar/Downloads/m
['valine', 'ceii', 'ciaciol', 'vii', ['kimi', 'karin']]
PS C:\Users\katar\Downloads\minggu11>
```

Gambar 11.9 : contoh penggunaan append.

b) extend = untuk menambahkan elemen pada sebuah list dan memperlakukan setiap elemen baru sebagai elemen list secara individual (tidak gabung menjadi satu kesatuan list).

```
laprak.py > ...
1  nama = ["valine", "ceii", "ciaciol", "vii"]
2  nama.extend(["kimi", "karin"])
3  print(nama)

PROBLEMS   OUTPUT   TERMINAL   ...   Python

PS C:\Users\katar\Downloads\minggu11> & C:/Users/katar/
pythoncore-3.14-64/python.exe c:/Users/katar/Downloads/
['valine', 'ceii', 'ciaciol', 'vii', 'kimi', 'karin']
PS C:\Users\katar\Downloads\minggu11>
```

Gambar 11.10 : contoh penggunaan extend.

c) sort = untuk mengurutkan elemen pada sebuah list.

```
laprak.py > ...
1  nama = ["valine", "ceii", "ciaciol", "vii"]
2  nama.sort()
3  print(nama)

PROBLEMS   OUTPUT   TERMINAL   ...   Python

PS C:\Users\katar\Downloads\minggu11> & C:/Users/kata
pythoncore-3.14-64/python.exe c:/Users/katar/Downloa
['ceii', 'ciaciol', 'valine', 'vii']
PS C:\Users\katar\Downloads\minggu11>
```

Gambar 11.11 : contoh penggunaan sort.

d) pop = untuk menghapus elemen pada sebuah list, apabila indeks elemen sudah diketahui dan ingin mendapatkan nilai yang dihapus.

```
laprak.py > ...
1  nama = ["valine", "ceii", "ciaciol", "vii"]
2  nama.pop(2)
3  print(nama)

PROBLEMS   OUTPUT   TERMINAL   ...   Python

PS C:\Users\katar\Downloads\minggu11> & C:/Users/kat
pythoncore-3.14-64/python.exe c:/Users/katar/Downloa
['valine', 'ceii', 'vii']
PS C:\Users\katar\Downloads\minggu11>
```

Gambar 11.12 : contoh penggunaan pop.

e) del = untuk menghapus elemen list tapi tidak perlu menampilkan elemen yang dihapus.

```
laprak.py > ...
1  nama = ["valine", "ceii", "ciaciol", "vii"]
2  del nama[1]
3  print(nama)

PROBLEMS   OUTPUT   TERMINAL   ...   Python

PS C:\Users\katar\Downloads\minggu11> & C:/Users/kata
pythoncore-3.14-64/python.exe c:/Users/katar/Downloa
['valine', 'ciaciol', 'vii']
PS C:\Users\katar\Downloads\minggu11>
```

Gambar 11.13 : contoh penggunaan del.

f) remove = untuk menghapus elemen list berdasarkan nilai tertentu.

```
laprak.py > ...
1  nama = ["valine", "ceii", "ciaciol", "vii"]
2  nama.remove("vii")
3  print(nama)

PROBLEMS   OUTPUT   TERMINAL   ...   Python

PS C:\Users\katar\Downloads\minggu11> & C:/Users/kat
pythoncore-3.14-64/python.exe c:/Users/katar/Downloa
['valine', 'ceii', 'ciaciol']
PS C:\Users\katar\Downloads\minggu11>
```

Gambar 11.14 : contoh penggunaan remove.

g) reverse = untuk membalik urutan list.

```
laprak.py > ...
1  nama = ["valine", "ceii", "ciaciol", "vii"]
2  nama.reverse()
3  print(nama)

PROBLEMS   OUTPUT   TERMINAL   ...   Python

PS C:\Users\katar\Downloads\minggu11> & C:/Users/kata
pythoncore-3.14-64/python.exe c:/Users/katar/Download
['vii', 'ciaciol', 'ceii', 'valine']
PS C:\Users\katar\Downloads\minggu11>
```

Gambar 11.15 : contoh penggunaan reverse.

Selain method tersebut, ada juga built-in yang dapat diakses oleh list, yaitu:

a) min() = untuk mengetahui nilai minimal dari elemen list (integer / angka).

```
laprak.py > ...
1  nomor = [23,25,7,5]
2  print(f"nilai minimumnya adalah {min(nomor)}")

PROBLEMS   OUTPUT   TERMINAL   ...   Python

PS C:\Users\katar\Downloads\minggu11> & C:/Users/katar/
pythoncore-3.14-64/python.exe c:/Users/katar/Downloads/
nilai minimumnya adalah 5
PS C:\Users\katar\Downloads\minggu11>
```

Gambar 11.16 : contoh penggunaan min.

b) max() = untuk mengetahui nilai maksimal dari elemen list (integer / angka).

```
laprak.py > ...
1  nomor = [23,25,7,5]
2  print(f"nilai maksimumnya adalah {max(nomor)}")

PROBLEMS   OUTPUT   TERMINAL   ...   Python

PS C:\Users\katar\Downloads\minggu11> & C:/Users/katar/Ap
pythoncore-3.14-64/python.exe c:/Users/katar/Downloads/mi
nilai maksimumnya adalah 25
PS C:\Users\katar\Downloads\minggu11>
```

Gambar 11.17 : contoh penggunaan max.

c) sum() = untuk mengetahui jumlah dari total elemen list (integer / angka).

```
laprak.py > ...
1  nomor = [23,25,7,5]
2  print(f"jumlahnya adalah {sum(nomor)}")

PROBLEMS   OUTPUT   TERMINAL   ...   Py

PS C:\Users\katar\Downloads\minggu11> & C:/Users
pythoncore-3.14-64/python.exe c:/Users/katar/Dow
jumlahnya adalah 60
PS C:\Users\katar\Downloads\minggu11>
```

Gambar 11.18 : contoh penggunaan sum.

d) len() = untuk mengetahui panjang list.

```
laprak.py > ...
1   nomor = [23,25,7,5]
2   print(f"panjang listnya adalah {len(nomor)}")

PROBLEMS   OUTPUT   TERMINAL   ...   Python

PS C:\Users\katar\Downloads\minggu11> & C:/Users/katar
pythoncore-3.14-64/python.exe c:/Users/katar/Downloads
panjang listnya adalah 4
PS C:\Users\katar\Downloads\minggu11>
```

Gambar 11.19 : contoh penggunaan len.

List juga dapat dilakukan list comprehension. Sehingga sintaks pada operasi list menjadi lebih singkat.

```
laprak.py > ...
1   nama = ["makan","malam", "nanas", "sekarang"]
2   baru = [x for x in nama if "an" in x]
3   print(baru)

PROBLEMS   OUTPUT   TERMINAL   ...   Python

PS C:\Users\katar\Downloads\minggu11> & C:/Users/katar
pythoncore-3.14-64/python.exe c:/Users/katar/Downloads
['makan', 'nanas', 'sekarang']
PS C:\Users\katar\Downloads\minggu11>
```

Gambar 11.20 : contoh penggunaan list comprehension untuk memeriksa apakah dalam elemen list mengandung substring “an”.

```
laprak.py > ...
1   nama = ["makan","malam", "nanas", "sekarang"]
2   baru = [x.upper() for x in nama]
3   print(baru)

PROBLEMS   OUTPUT   TERMINAL   ...   Python

PS C:\Users\katar\Downloads\minggu11> & C:/Users/katar/
pythoncore-3.14-64/python.exe c:/Users/katar/Downloads/
['MAKAN', 'MALAM', 'NANAS', 'SEKARANG']
PS C:\Users\katar\Downloads\minggu11>
```

Gambar 11.21 : contoh penggunaan list comprehension untuk mengubah semua elemen list menjadi huruf besar.

```
laprak.py > ...
1   nama = ["makan","malam", "nanas", "sekarang"]
2   baru = [x for x in nama if len(x) > 5]
3   print(baru)

PROBLEMS   OUTPUT   TERMINAL   ...   Python

PS C:\Users\katar\Downloads\minggu11> & C:/Users/katar/
pythoncore-3.14-64/python.exe c:/Users/katar/Downloads/
['sekarang']
PS C:\Users\katar\Downloads\minggu11>
```

Gambar 11.22 : contoh penggunaan list comprehension untuk memeriksa dan mengambil elemen list yang panjangnya lebih dari 5 huruf.

4. List Sebagai Parameter Fungsi

Apabila list digunakan sebagai parameter fungsi maka sifatnya juga **mutable**. Apa yang diubah dalam fungsi akan mengubah nilai aslinya juga.

```
laprak.py > ...
def ubah_data(x):
    x[0] = "valine"

x = ["k", "a", "r", "i", "n"]
data = ubah_data(x)
print(x)

PROBLEMS    OUTPUT    TERMINAL    ...

PS C:\Users\katar\Downloads\minggu11> & C
pythoncore-3.14-64/python.exe c:/Users/ka
['valine', 'a', 'r', 'i', 'n']
PS C:\Users\katar\Downloads\minggu11>
```

Gambar 11.23 : contoh penggunaan list sebagai parameter fungsi yang akan mengubah nilai aslinya.

Kelebihan menggunakan tipe data list sebagai parameter :

a) **Fleksibilitas** = fungsi dapat memproses berbagai jenis data dengan menggunakan list.
b) **Reusabilitas** = fungsi dapat digunakan kembali dengan list yang berbeda.
c) **Pembagian tugas** = list dapat membantu membagi tugas menjadi bagian-bagian yang lebih kecil.
d) **Pengorganisasian data** = list dapat membantu mengorganisir data sebelum diproses oleh fungsi.

```
laprak.py > ...
def hapus_data(x):
    return data[1:]

data = ["valine", "b", "c"]
hasil = hapus_data(data)
print(hasil)

PROBLEMS    OUTPUT    TERMINAL    ...

PS C:\Users\katar\Downloads\minggu11> &
pythoncore-3.14-64/python.exe c:/Users/
['b', 'c']
PS C:\Users\katar\Downloads\minggu11>
```

Gambar 11.24 : contoh penggunaan list sebagai parameter fungsi yang akan menghapus elemen pertama list.

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github : https://github.com/angelarevalines/71251188_valine.git

SOAL 1

Source Code :

```
def tiga_terbaik(data):
    urut = sorted(data, reverse=True)
    bilangan = urut[:3]
    return f"3 bilangan terbaik dari {data} adalah {bilangan}"

print(tiga_terbaik([3,4,8,3,9,40,53,2,6,23]))
print(tiga_terbaik([43,89,9,2,4,7,18,37,22]))
```

Penjelasan :

1. Buat fungsi bernama **tiga_terbaik** dengan parameter **data** yang akan berfungsi untuk mengurutkan data yang ada dan mencari 3 bilangan tertingginya.
2. Membuat variabel bernama **urut** yang akan menyimpan data yang telah diurutkan dari yang terbesar ke terkecil.
3. Membuat variabel bernama **bilangan** yang menyimpan 3 bilangan terbesar pertama dari data.
4. Mengembalikan nilai dari fungsi sesuai output yang diinginkan.
5. Mencoba test case dengan memanggil nama fungsinya.

Output :

```
3 bilangan terbaik dari [3, 4, 8, 3, 9, 40, 53, 2, 6, 23] adalah [53, 40, 23]
3 bilangan terbaik dari [43, 89, 9, 2, 4, 7, 18, 37, 22] adalah [89, 43, 37]
```

SOAL 2

Source Code :

```
def average():
    hasil = []
      print('Masukkan bilangan-bilangan dan ketik "done" jika sudah selesai')


    while True:
        angka = input("Bilangan = ")
        if angka.lower() == "done":
            break
        try:
            bilangan = float(angka)
            hasil.append(bilangan)
        except:
            print("Masukkan angka yang valid!")


    if len(hasil) > 0:
        ratarata = sum(hasil) / len(hasil)
            print(f"rata-rata dari {len(hasil)} bilangan yang dimasukkan adalah {ratarata:.2f}")
    else:
        print("Tidak ada bilangan yang kamu masukkan")


average()
```

Penjelasan :

1. Membuat fungsi bernama **average** untuk menghitung rata-rata dari semua nilai yang diinputkan user.
2. Membuat variabel bernama **hasil** yang berupa list kosong yang akan menyimpan semua angka yang nanti dimasukkan oleh user.
3. Menggunakan **print** untuk menampilkan output.

4. Menggunakan fungsi perulangan **while** yang hanya akan berhenti apabila user mengetik “done”.
5. Membuat variabel bernama **angka** yang akan digunakan untuk meminta user menginput angka.
6. Menggunakan try-except untuk menghindari error apabila user salah input.
7. Angka diubah menjadi angka desimal, lalu disimpan dalam variabel **bilangan** yang setelah itu akan dimasukkan ke dalam list hasil dengan menggunakan **append()**.
8. Sedangkan jika input bukan berupa angka maka keluarkan output “Masukkan angka yang valid!”
9. Setelah perulangan while selesai maka kita cek apakah ada angka yang dimasukkan atau tidak, jika ada lalu hitung rata-ratanya menggunakan sum() / len().
10. Jika tidak maka keluarkan output “Tidak ada bilangan yang kamu masukkan.”
11. Panggil fungsi agar program dapat berjalan.

Output :

```
Masukkan bilangan-bilangan dan ketik "done" jika sudah selesai
Bilangan = 80
Bilangan = 99
Bilangan = 84
Bilangan = 87
Bilangan = 90
Bilangan = 89
Bilangan = 95
Bilangan = done
rata-rata dari 7 bilangan yang dimasukkan adalah 89.14
```

SOAL 3

Source Code :

```
namafile = "alpro.txt"
file = open(namafile).read()

teks = file.lower()

tanda = "!()-[]}{:;.,'?!@#$%^&*_+"
for t in tanda:
    teks = teks.replace(t, "")
```

```
kata = teks.split()
unik = set(kata)
print("Kata yang muncul lebih dari 1 kali:")

for k in unik:
    jumlah = kata.count(k)
    if jumlah > 1:
        print(f"{k} = {jumlah} kali")
```

Penjelasan :

1. Buat file text bernama **alpro (alpro.txt)** yang nantinya akan kita cek.
2. Buka dan baca seluruh isi file menjadi satu string panjang lalu simpan dalam variabel bernama **file**.
3. Membuat variabel bernama **teks** yang akan menyimpan isi file yang telah diubah menjadi huruf kecil semua menggunakan **lower()**.
4. Simpan semua tanda baca yang ingin dihapus dalam variabel **tanda**.
5. Lalu menggunakan perulangan for yang akan membaca setiap karakter tanda baca lalu menghapusnya.
6. Buat variabel bernama **kata** yang akan menyimpan isi teks yang telah dipisahkan berdasarkan spasi menggunakan **split()**.
7. Buat variabel bernama **unik** yang akan menyimpan kata tanpa ada kata yang sama menggunakan **set()**.
8. Lalu hitung menggunakan **count()** dan simpan dalam variabel **jumlah**.
9. Tampilkan kata yang muncul lebih dari 1x menggunakan **print**.

Output :

```
Kata yang muncul lebih dari 1 kali:
krim = 3 kali
makan = 2 kali
aku = 2 kali
es = 3 kali
```